

PA1: From Bellman Updates to TD Control

<Your name> · <Student ID> · Reinforcement Learning (Graduate)

Implementation details, conventions, seeds and reproduction commands are in README.md. All graded numbers come from results.json (spec settings, seeds 0–4). Experiments marked [supp.] are supplementary: they reuse the same update rules and agent.epsilon_greedy (a check confirms bit-identical Q tables to agent.sarsa/agent.q_learning) and use 20 seeds where stated (python extra_experiments.py).

1 Policy iteration vs value iteration

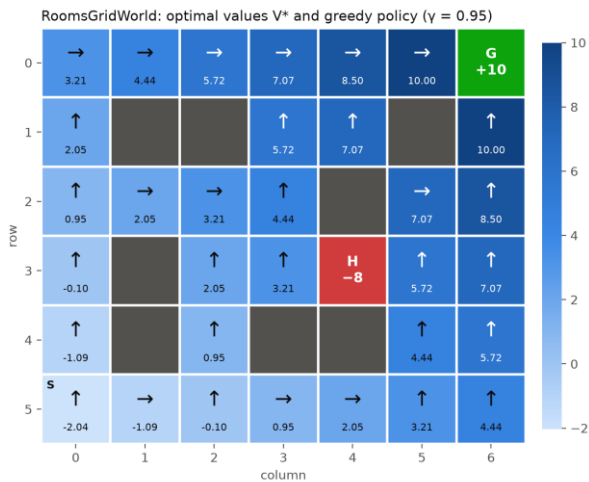


Figure 1. RoomsGridWorld: V^* (colour, value in cell) and the greedy policy (arrows). PI and VI produce exactly this map.

Both methods reach the same optimal policy and the same values.

The largest difference between V_{PI} and V_{VI} is exactly 0.0, every state gets the same action, and the greedy path from S (up the left column, then right along the top row) reaches G in 11 steps with undiscounted return 0 ($V^*(S) = -2.038$). At S, *up* and *right* are exactly tied ($Q = -2.0379$ for both); the lowest-index rule selects *up*.

The counts differ because they count different things. PI needs **9 rounds**, VI **12 sweeps**, but every PI round contains a complete policy evaluation. Counted in sweeps, PI does **3,276 evaluation sweeps** ($6 \times 540 + 3 \times 12$) versus 12 for VI, about 270 times more backups (Table 1, Figure 2).

	Policy iteration	Value iteration
Iterations	9 rounds	12 sweeps
Total sweeps	3,276 (+9 improv.)	12
$V^*(S)$, action at S	-2.038, up	-2.038, up
$\max V_{PI} - V_{VI} $	0.0	0.0

Table 1. DP summary (values identical to results.json).

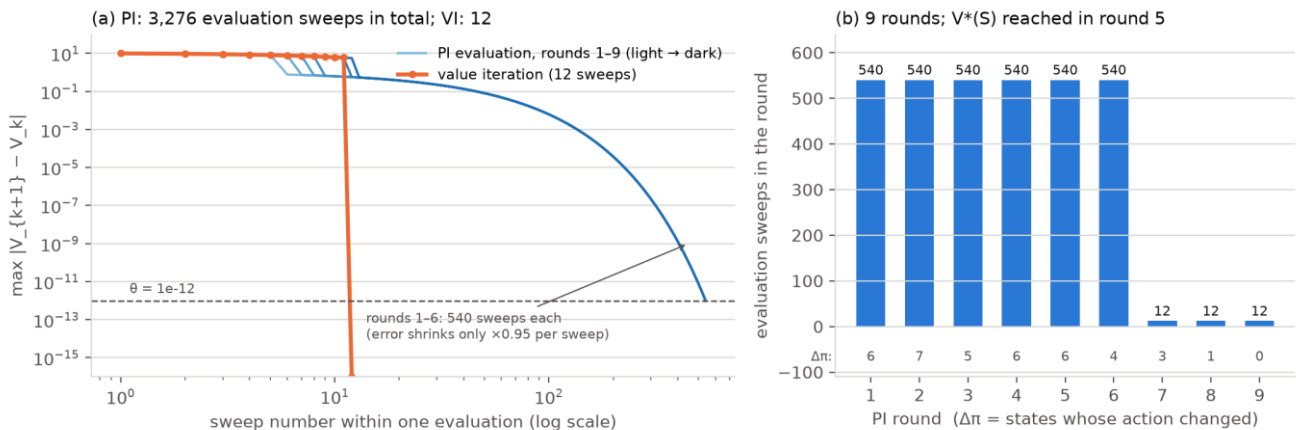


Figure 2 [supp.]. (a) Largest value change per sweep. Each light-to-dark blue curve is one PI evaluation; orange is VI. (b) Evaluation sweeps per PI round and the number of states whose action changed ($\Delta\pi$).

Why PI rounds 1–6 cost 540 sweeps. The initial all-*up* policy makes most states bump into a wall or the top border forever, so their value is $-1/(1-\gamma) = -20$. For a fixed policy, the evaluation error shrinks only by a factor $\gamma = 0.95$ per sweep, and reaching $\theta = 10^{-12}$ needs $0.95^k < 10^{-12}$, i.e. $k \approx 539$, which is exactly the 540 observed. Each improvement step only re-routes the few states next to already-good states ($\Delta\pi = 4-7$), so $V(S)$ stays at -20 until round 5. From round 7 every state's path terminates, and evaluation of a deterministic, terminating policy is exact after 12 sweeps. **Why VI needs only 12.** VI applies the max in every sweep, so correct values spread outward from G by one cell per sweep: the largest change follows 10×0.95^k (10, 9.5, 9.03, ..., 5.99). S is the farthest state at 11 steps, so after 11 sweeps every value is exact and sweep 12 confirms $\Delta = 0$. PI takes few large steps (policy improvement is a Newton-like jump), each paid for by a full evaluation; VI takes many cheap steps. On this small deterministic grid, VI is far cheaper.

2 The paths learned by SARSA and Q-learning

Q-learning learns the shortest path along the cliff edge; SARSA learns a detour through the top rows. Q-learning's greedy path runs along row 3, directly above the cliff, on every seed: 11 steps, return -11 , **highest_row = 3.0**, identical to the DP optimum. SARSA's path climbs to the top of the grid before crossing: 15–17 steps, return -16.6 , **highest_row = 0.2** (four seeds reach row 0, one row 1). With 20 seeds the picture is the same (Q-learning 3.0; SARSA rows 0–1 only).

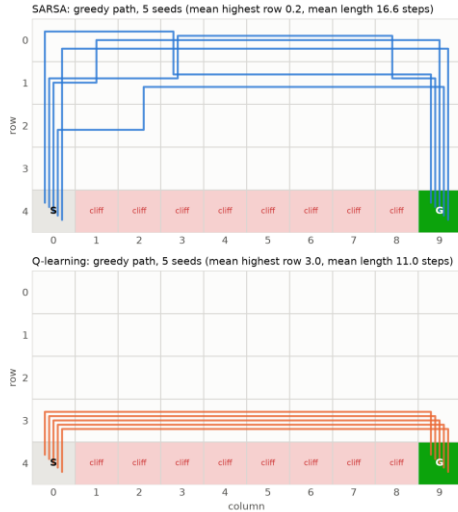


Figure 3. Greedy path of every seed (0–4), slightly offset. Q-learning: identical on all seeds.

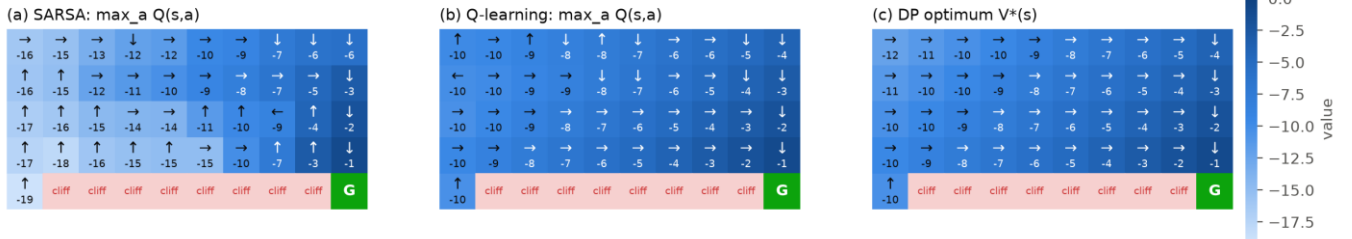


Figure 4 [supp.]. Learned state values $\max_a Q(s,a)$ (mean of seeds 0–4) with the greedy action of seed 0, next to the DP optimum. Q-learning recovers V^* ; SARSA's values drop sharply near the cliff.

3 The effect of ϵ

We trained both algorithms with $\epsilon \in \{0.01, 0.05, 0.1, 0.2, 0.3\}$, everything else fixed (5 seeds in results_sweep/, repeated with 20 seeds [supp.]; the conclusions agree, and 20-seed numbers are reported). Besides the three requested quantities we measured **exploration cost** (share of training episodes containing a cliff fall) and **greedy stability**: at the end of each of the last 500 training episodes, does the greedy policy at that moment reach G?

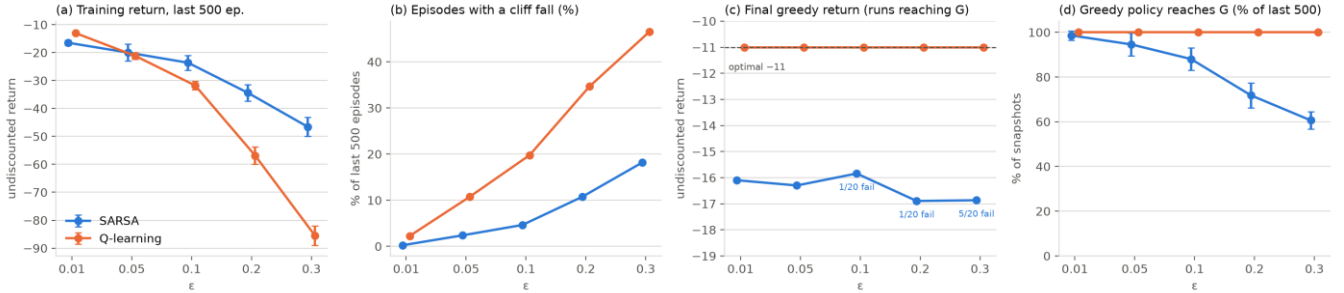


Figure 5 [supp.]. ϵ sweep, 20 seeds, mean ± 1 std. (c) Final greedy return over runs that reach G; annotations count runs whose final greedy policy never reaches G. (d) Share of late-training greedy snapshots that reach G.

ϵ	Training return S / Q	Episodes with fall S / Q	Final greedy S / Q	SARSA greedy reaches G	SARSA highest row (median)
0.01	-16.5 / -13.0	0.2 % / 2.3 %	-16.1 / -11	20/20 · 98.6 %	0
0.05	-20.0 / -21.2	2.4 % / 10.7 %	-16.3 / -11	20/20 · 94.6 %	0
0.1	-23.6 / -31.8	4.6 % / 19.8 %	-15.8 / -11	19/20 · 88.0 %	1
0.2	-34.4 / -56.8	10.7 % / 34.6 %	-16.9 / -11	19/20 · 71.8 %	0
0.3	-46.6 / -85.5	18.2 % / 46.5 %	-16.9 / -11	15/20 · 60.6 %	0

Table 2 [supp.]. 20 seeds; S = SARSA, Q = Q-learning; bold = better training return. Q-learning's greedy policy reaches G on 20/20 seeds and in 100 % of snapshots at every ϵ . Final greedy S averages runs that reach G; failures are analysed in Table 4.

Exploration. Larger ϵ means more random actions and more falls, but the cost depends on where the agent walks: Q-learning, on the edge, falls in 2.3 % $\rightarrow$ 46.5 % of episodes as ϵ goes from 0.01 to 0.3; SARSA, away from the edge, in only 0.2 % $\rightarrow$ 18.2 %. **Training return** therefore drops for both, much faster for Q-learning ($-13.0 \rightarrow -85.5$ vs $-16.5 \rightarrow -46.6$). The ranking even reverses: at $\epsilon = 0.01$ Q-learning trains better (-13.0 vs -16.5), at 0.05 they are equal, and from 0.1 on SARSA is better (Figure 6). **Final greedy evaluation** separates the methods sharply. Q-learning's greedy policy is optimal (-11) at every ϵ and seed: its target does not depend on ϵ , and in this deterministic task its estimates converge to q^* even with a constant α . SARSA's greedy path stays on the safe detour at every ϵ , and it becomes **less reliable** as ϵ grows: the share of late snapshots whose greedy policy reaches G falls from 98.6 % to 60.6 %, and 5 of 20 final policies fail at $\epsilon = 0.3$.

Why. The two update targets estimate different things. Q-learning's target $r + \gamma \max Q(s', \cdot)$ assumes the agent acts greedily from the next step on, so it learns q^* , the value of the optimal policy, regardless of the exploration it actually performs (Bellman optimality). SARSA's target $r + \gamma Q(s', a')$ uses the action the ϵ -greedy policy actually takes, so it learns q_π for the ϵ -greedy policy itself (Bellman expectation). On row 3 every step has a $0.1 \times \frac{1}{4} = 2.5$ % chance of a random *down* move: -75 and a restart at S. SARSA's values price that risk in; Q-learning's do not.

Figure 4 makes this visible: along row 3 the learned value is **-12.5 for SARSA** but **-5.4 for Q-learning** ($= V^*$ exactly), while SARSA rates the top row higher (-10.6), so its greedy policy goes up.

How far up also depends on α . The exact point SARSA converges to, q_π^* (ϵ the optimal ϵ -greedy values, computed from the model), keeps only one row of margin at $\epsilon = 0.1$: row 2, 13 steps, -13 . With $\alpha = 0.1$ all 20 seeds learn exactly that path. With the spec's $\alpha = 0.5$ each fall (-75) pulls an estimate halfway towards one catastrophic target, so values near the edge stay too low and SARSA overshoots to rows 0–1 (Table 4).

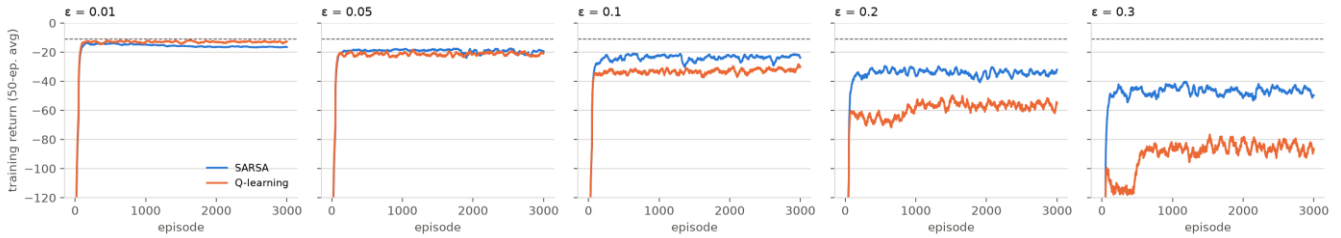


Figure 6 [supp.]. Training return (50-episode moving average, 20 seeds) per ϵ . Dashed: optimal -11 . The SARSA–Q-learning gap opens as ϵ grows.

Are these failures a bug? The spec warns that a greedy policy that fails to reach G indicates a bug *at convergence*, so we tested whether SARSA had converged (Table 4). (i) Its convergence target q^*_ϵ , solved exactly from the model, has a greedy policy that reaches G at every ϵ . (ii) On the same seeds, a smaller ($\alpha = 0.1$) or decaying step size removes **every** failure (0/20 at all ϵ) and makes the greedy policy stable in 98–100 % of snapshots, and $\alpha = 0.1$ lands exactly on q^*_ϵ 's path at $\epsilon = 0.1$. (iii) The code matches a reference SARSA bit for bit. With $\alpha = 0.5$, SARSA's target samples a' from the exploring policy (a random a' near the cliff is worth about -100) and each sample moves Q halfway, so Q never settles and near-tied actions keep swapping, which occasionally creates a loop (e.g. $\epsilon = 0.2$, seed 3: $Q(S, \text{up}) = -24.02 < Q(S, \text{down}) = -23.85$, and *down* bumps the border). The failures are therefore **non-convergence under the prescribed constant α** , not an implementation error. Q-learning is unaffected because its max-target involves no sampled action, so in this deterministic task it converges to q^* even with $\alpha = 0.5$. All graded runs ($\epsilon = 0.1$, seeds 0–4) reach G .

ϵ	Converged target q^*_ϵ : path	SARSA $\alpha = 0.5$ (spec): fails · stable · rows	SARSA $\alpha = 0.1$: fails · stable · rows	SARSA $\alpha = 0.5/(1+n/50)$: fails · stable · rows
0.01	row 3, -11 (edge)	0/20 · 98.6 % · 0–1	0/20 · 99.7 % · 2–3	0/20 · 100 % · 1–2
0.1	row 2, -13	1/20 · 88.0 % · 0–1	0/20 · 99.8 % · 2 (all)	0/20 · 100 % · 1–2
0.2	row 2, -13	1/20 · 71.8 % · 0–1	0/20 · 99.5 % · 1–2	0/20 · 100 % · 1
0.3	row 1, -15	5/20 · 60.6 % · 0–1	0/20 · 97.9 % · 0–2	0/20 · 100 % · 1

Table 4 [supp.]. Diagnosis of SARSA's greedy failures (20 seeds, 3,000 episodes). fails = final greedy policy never reaches G ; stable = share of the last 500 greedy snapshots that reach G ; rows = highest rows of the final greedy paths. n = visits to (s,a) .

Small ϵ has a cost too: too little exploration. At $\epsilon = 0.01$ the edge path is also optimal for SARSA's own objective: q^*_ϵ is the edge path (Table 4), and executing Q-learning's policy with $\epsilon = 0.01$ yields -12.8 versus -17.0 for SARSA's (Figure 8). Yet SARSA does not find it: in a 20,000-episode run at $\epsilon = 0.01$ (Figure 9c), the greedy path drifts from row 0 to rows 1–2 at best (one seed reaches row 2 and -13) but never reaches the edge. Early episodes, with Q initialised at 0, fall off the cliff many times, so the edge actions start with very negative values; with $\epsilon = 0.01$ they are almost never tried again, so those early estimates are not corrected (even with $\alpha = 0.1$, only 1 of 20 seeds reaches the edge). **In short, ϵ trades training cost against discovery and, for SARSA, against greedy reliability.**

4 Worse training return, better greedy return

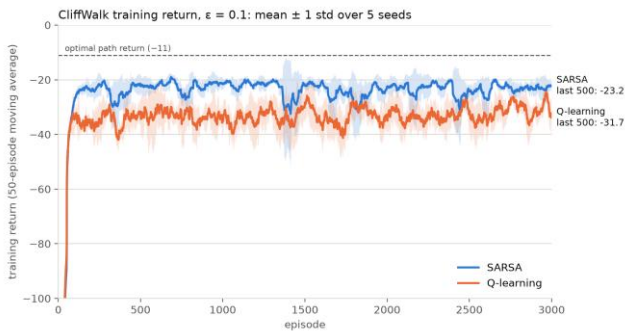


Figure 7. Training return at $\epsilon = 0.1$, mean ± 1 std over seeds 0–4 (50-episode moving average; y-axis clipped at -100). Labels: raw last-500 means, as in results.json.

Both facts hold because each algorithm optimises the return of a different policy. At the spec settings ($\epsilon = 0.1$, seeds 0–4), Q-learning's training return is worse (-31.7 vs -23.2) while its greedy return is better (-11 vs -16.6). The training return measures the **ϵ -greedy behaviour policy while it learns**; the greedy return measures the **learned policy with exploration switched off**.

To separate the two effects we froze each learned Q and executed its policy, without learning, at different ϵ (Table 3, Figure 8). Executed with $\epsilon = 0.1$, SARSA's policy earns -20.9 and Q-learning's -31.7 , which is the same as the DP-optimal policy executed ϵ -greedily (-31.8). Executed greedily, the order flips (-16.6 vs -11.0). The lines cross near $\epsilon \approx 0.04$.

	SARSA	Q-learning
Training return ($\epsilon = 0.1$, learning)	-23.2	-31.7
Policy run at $\epsilon = 0.1$, no learning	-20.9	-31.7
Cliff falls per episode at $\epsilon = 0.1$	0.03	0.24
Greedy return ($\epsilon = 0$)	-16.6	-11.0
highest_row	0.2	3.0

Table 3. Seeds 0–4; frozen-policy rows [supp.] are averages of 2,000 episodes per seed.

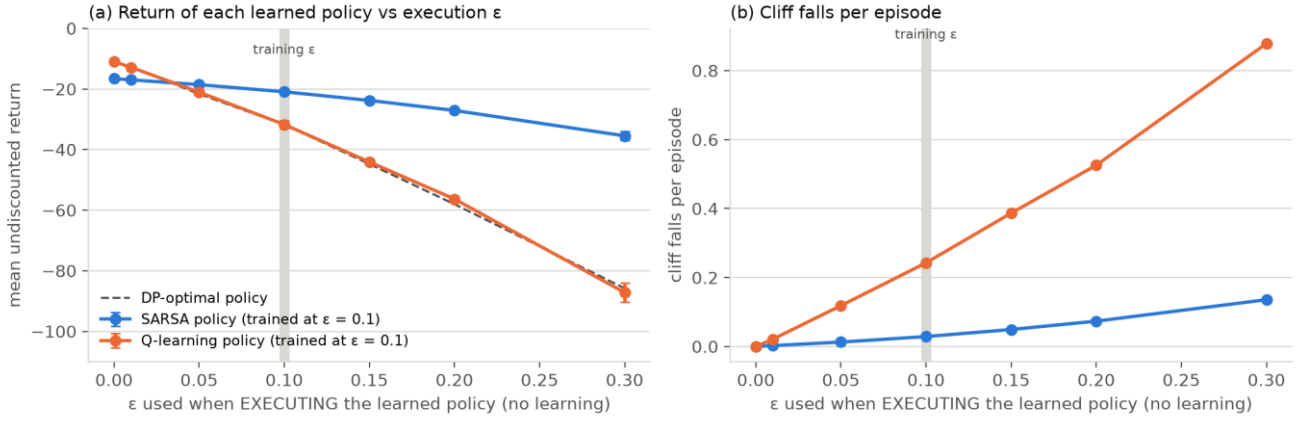


Figure 8 [supp.]. Frozen policies (trained at $\epsilon = 0.1$) executed at different ϵ without learning. (a) Return; dashed: DP-optimal policy. (b) Cliff falls per episode.

What each algorithm is estimating. Q-learning is off-policy: it estimates q^* , the value of acting optimally, which assumes no future random moves. Its greedy policy is therefore optimal once exploration stops, but during training that policy is executed ϵ -greedily on the most dangerous route, with about 0.24 cliff falls per episode. SARSA is on-policy: it estimates q_π of the ϵ -greedy policy it actually runs, including its random moves. It therefore converges towards the policy that is best *given that it keeps exploring*, which is a safer route with fewer falls and a better training return, but not the greedy optimum. Neither is wrong; they answer different questions, and the right choice depends on whether exploration continues when the policy is used.

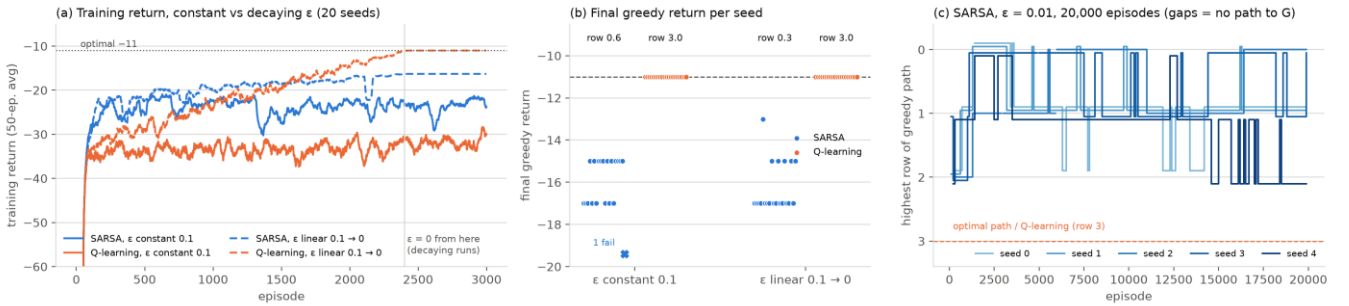


Figure 9 [supp.]. (a), b) ϵ decayed linearly from 0.1 to 0 over the first 2,400 of 3,000 episodes, 20 seeds. Labels in (b): mean highest row. (c) SARSA's highest row during a 20,000-episode run at $\epsilon = 0.01$ (gaps: greedy policy does not reach G).

Removing exploration removes the training gap. With ϵ decayed to 0 (Figure 9), Q-learning's training return converges to the optimum -11 , so training return equals greedy return. SARSA's also equals its greedy return (-16.3 , 20/20 seeds stable) but stays on the safe detour (highest row 0.35). Theory guarantees SARSA reaches the optimum only under GLIE exploration *and* decreasing step sizes (Singh et al., 2000). With a constant $\alpha = 0.5$ and a finite budget, its early pessimism about the edge is locked in, as the 20,000-episode run also shows.

5 Conclusions

- **DP:** PI and VI give identical V^* and policies. VI needs 12 sweeps; PI needs 9 rounds but 3,276 evaluation sweeps, because early policies loop and evaluation converges only at rate γ .
- **On- vs off-policy:** Q-learning learns q^* and the edge path (highest_row 3, return -11). SARSA learns the value of its ϵ -greedy behaviour, prices in the 2.5 % per-step fall risk, and takes a detour (highest_row ≈ 0 , -16.6). Its converged target keeps one row of margin (row 2, -13); the extra distance comes from the large constant α .
- **ϵ :** higher ϵ raises falls and lowers training return, far more for Q-learning, and makes SARSA's greedy policy unstable (98.6 % $\rightarrow$ 60.6 % reliable). This is non-convergence under $\alpha = 0.5$, not a bug: with a smaller or decaying α , failures drop to 0/20. Very small ϵ reverses the training-return ranking and starves SARSA of the exploration it needs to revise early estimates.
- **Training vs greedy:** SARSA wins while exploring (-20.9 vs -31.7 at $\epsilon = 0.1$), Q-learning wins without exploration (-11.0 vs -16.6). The crossover is at $\epsilon \approx 0.04$.

Limitations. Graded results use 5 seeds (the supplementary runs use 20). The prescribed constant $\alpha = 0.5$ prevents SARSA from converging, so its final greedy policy is a snapshot of a moving estimate (Table 4). All findings are from one small deterministic environment. *Reference:* Sutton & Barto (2018), *Reinforcement Learning: An Introduction*, 2nd ed., Ex. 6.6; Singh, Jaakkola, Littman & Szepesvári (2000), *Machine Learning* 38:287–308.